

SHETH L.U.J & SIR MV COLLEGE

Aim:Digital Signature

Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

Code:CLI

```
import hashlib
import secrets
import math

def is_prime(n, k=10):
    if n < 2:
        return False
    small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
    for p in small_primes:
        if n == p:
            return True
        if n % p == 0:
            return False

    d = n - 1
    r = 0
    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(k):
        a = secrets.randbelow(n - 3) + 2
        x = pow(a, d, n)

        if x == 1 or x == n - 1:
            continue

        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
```

SHETH L.U.J & SIR MV COLLEGE

```
return True
```

```
def generate_prime(bits):  
    while True:  
        n = secrets.randbits(bits)  
        n |= (1 << bits - 1) | 1  
        if is_prime(n):  
            return n
```

```
def generate_keys():  
    p = generate_prime(256)  
    q = generate_prime(256)  
  
    while p == q:  
        q = generate_prime(256)
```

```
    n = p * q  
    phi = (p - 1) * (q - 1)
```

```
    e = 65537
```

```
    while math.gcd(e, phi) != 1:  
        e += 2
```

```
    d = pow(e, -1, phi)
```

```
    public_key = (e, n)  
    private_key = (d, n)
```

```
    return public_key, private_key
```

```
def hash_message(message):  
    return int.from_bytes(  
        hashlib.sha256(message.encode()).digest(),  
        byteorder="big"  
    )
```

```
def sign_message(message, private_key):  
    d, n = private_key  
    h = hash_message(message)  
    signature = pow(h, d, n)  
    return signature
```

```
def verify_signature(message, signature, public_key):
```

SHETH L.U.J & SIR MV COLLEGE

```
e, n = public_key
h = hash_message(message)
recovered_hash = pow(signature, e, n)
return h == recovered_hash
```

```
def main():
    print("T071 DIGITAL SIGNATURE")
    print("=" * 40)

    public_key, private_key = generate_keys()

    print("\nPublic Key:")
    print("e =", public_key[0])
    print("n =", public_key[1])

    print("\nPrivate Key:")
    print("d =", private_key[0])
    print("n =", private_key[1])

    message = input("\nEnter message: ")

    signature = sign_message(message, private_key)

    print("\nDigital Signature:")
    print(signature)

    result = verify_signature(message, signature, public_key)

    print("\nVerification Result:")
    if result:
        print("Signature is VALID")
        print("Message integrity and authenticity verified.")
    else:
        print("Signature is INVALID")

    choice = input("\nDo you want to test a modified message? (y/n): ")

    if choice.lower() == "y":
        modified_message = input("Enter modified message: ")

        result = verify_signature(
            modified_message,
            signature,
            public_key
```

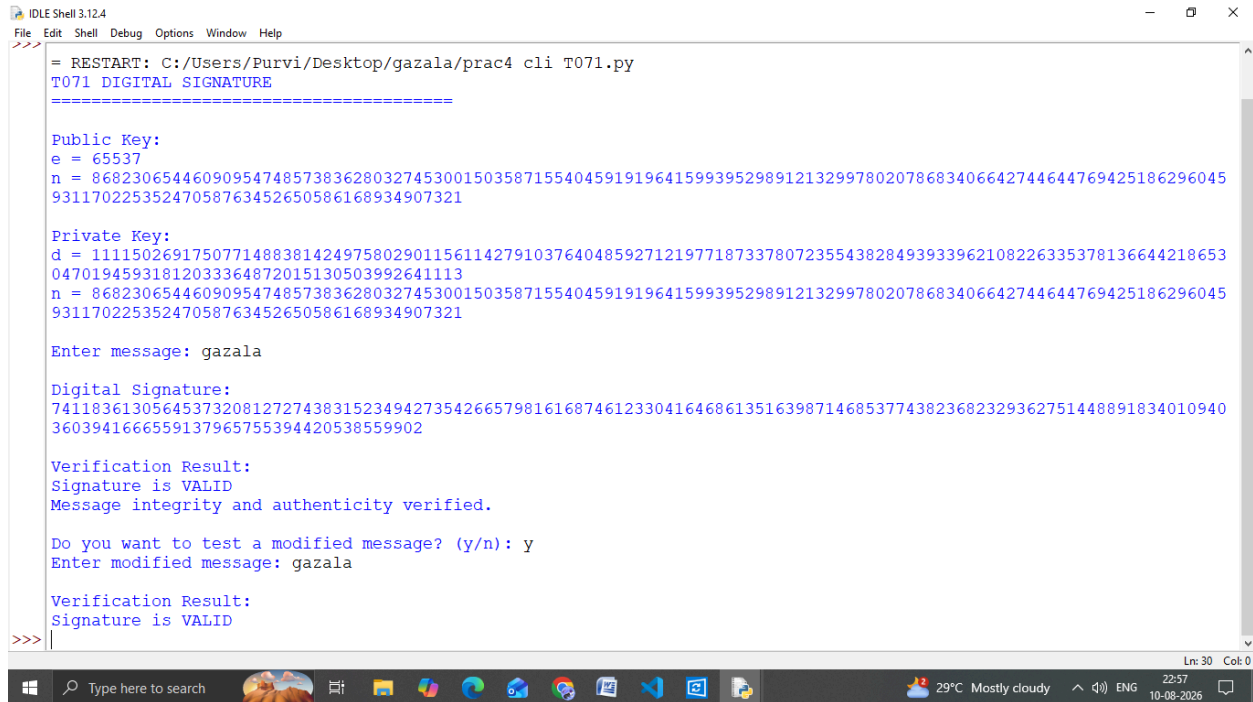
SHETH L.U.J & SIR MV COLLEGE

```
)

print("\nVerification Result:")
if result:
    print("Signature is VALID")
else:
    print("Signature is INVALID")
    print("Message has been modified.")

if __name__ == "__main__":
    main()
```

Output:valid



```
IDLE Shell 3.12.4
File Edit Shell Debug Options Window Help

= RESTART: C:/Users/Purvi/Desktop/gazala/prac4 cli T071.py
T071 DIGITAL SIGNATURE
=====

Public Key:
e = 65537
n = 86823065446090954748573836280327453001503587155404591919641599395298912132997802078683406642744644769425186296045
93117022535247058763452650586168934907321

Private Key:
d = 11115026917507714883814249758029011561142791037640485927121977187337807235543828493933962108226335378136644218653
04701945931812033364872015130503992641113
n = 86823065446090954748573836280327453001503587155404591919641599395298912132997802078683406642744644769425186296045
93117022535247058763452650586168934907321

Enter message: gazala

Digital Signature:
741183613056453732081272743831523494273542665798161687461233041646861351639871468537743823682329362751448891834010940
3603941666559137965755394420538559902

Verification Result:
Signature is VALID
Message integrity and authenticity verified.

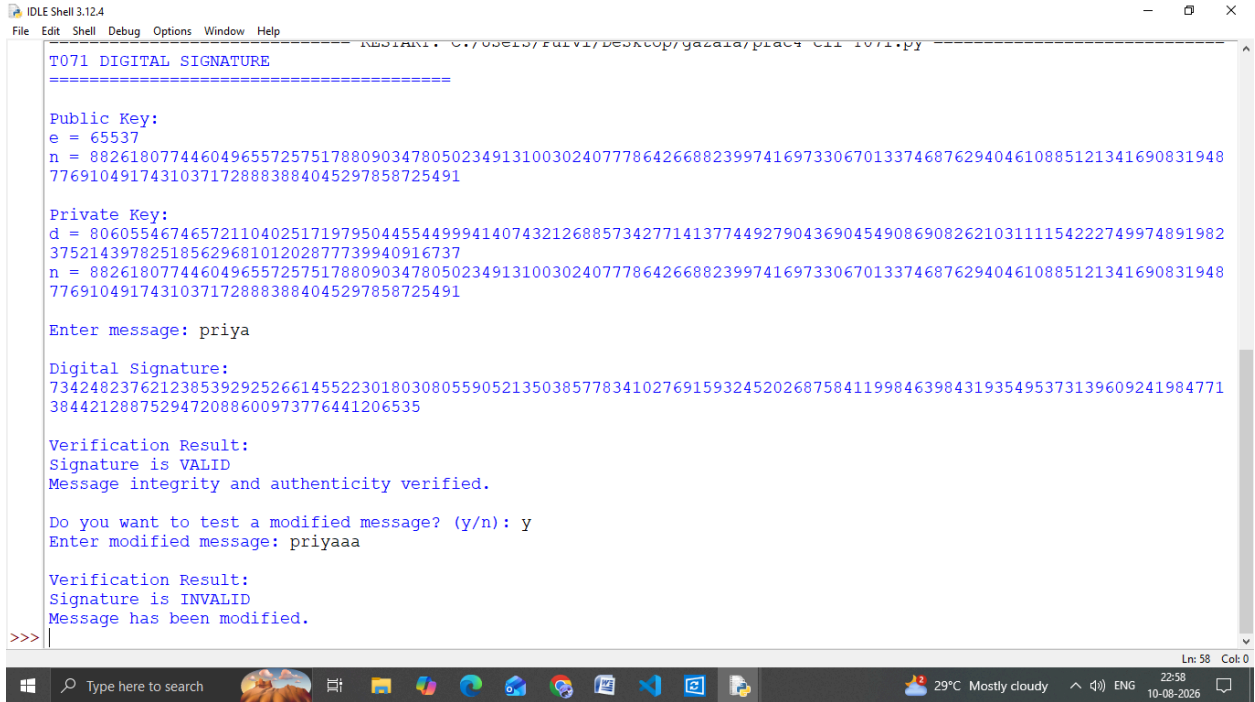
Do you want to test a modified message? (y/n): y
Enter modified message: gazala

Verification Result:
Signature is VALID

>>>
```

SHETH L.U.J & SIR MV COLLEGE

invalid



```
-----BEGIN-----
T071 DIGITAL SIGNATURE
=====

Public Key:
e = 65537
n = 88261807744604965572575178809034780502349131003024077786426688239974169733067013374687629404610885121341690831948
77691049174310371728883884045297858725491

Private Key:
d = 80605546746572110402517197950445544999414074321268857342771413774492790436904549086908262103111154222749974891982
37521439782518562968101202877739940916737
n = 88261807744604965572575178809034780502349131003024077786426688239974169733067013374687629404610885121341690831948
77691049174310371728883884045297858725491

Enter message: priya

Digital Signature:
734248237621238539292526614552230180308055905213503857783410276915932452026875841199846398431935495373139609241984771
3844212887529472088600973776441206535

Verification Result:
Signature is VALID
Message integrity and authenticity verified.

Do you want to test a modified message? (y/n): y
Enter modified message: priyaaa

Verification Result:
Signature is INVALID
Message has been modified.
>>>
```

Code:GUI

```
import tkinter as tk
from tkinter import messagebox
import hashlib
import secrets
import math

def is_prime(n, k=10):
    if n < 2:
        return False

    small_primes = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

    for p in small_primes:
        if n == p:
            return True
        if n % p == 0:
            return False
```

GAZALA ALAM
T071

SHETH L.U.J & SIR MV COLLEGE

```
d = n - 1
```

```
r = 0
```

```
while d % 2 == 0:
```

```
    r += 1
```

```
    d //= 2
```

```
for _ in range(k):
```

```
    a = secrets.randbelow(n - 3) + 2
```

```
    x = pow(a, d, n)
```

```
    if x == 1 or x == n - 1:
```

```
        continue
```

```
    for _ in range(r - 1):
```

```
        x = pow(x, 2, n)
```

```
    if x == n - 1:
```

```
        break
```

```
    else:
```

```
        return False
```

```
return True
```

```
def generate_prime(bits):
```

```
    while True:
```

```
        n = secrets.randbits(bits)
```

```
        n |= (1 << bits - 1) | 1
```

```
    if is_prime(n):
```

```
        return n
```

```
def generate_keys():
```

```
    p = generate_prime(256)
```

```
    q = generate_prime(256)
```

```
    while p == q:
```

```
        q = generate_prime(256)
```

```
    n = p * q
```

```
    phi = (p - 1) * (q - 1)
```

```
    e = 65537
```

GAZALA ALAM

T071

SHETH L.U.J & SIR MV COLLEGE

```
while math.gcd(e, phi) != 1:  
    e += 2
```

```
d = pow(e, -1, phi)
```

```
return (e, n), (d, n)
```

```
def hash_message(message):  
    return int.from_bytes(  
        hashlib.sha256(message.encode()).digest(),  
        "big"  
    )
```

```
def sign_message(message, private_key):  
    d, n = private_key  
    h = hash_message(message)  
    return pow(h, d, n)
```

```
def verify_signature(message, signature, public_key):  
    e, n = public_key  
    h = hash_message(message)  
    recovered_hash = pow(signature, e, n)  
    return h == recovered_hash
```

```
public_key = None  
private_key = None  
signature = None
```

```
def generate():  
    global public_key, private_key, signature  
  
    public_key, private_key = generate_keys()  
    signature = None  
  
    public_key_box.delete("1.0", tk.END)  
    private_key_box.delete("1.0", tk.END)  
    signature_box.delete("1.0", tk.END)  
  
    public_key_box.insert(  
        tk.END,  
        f"e = {public_key[0]}\nn = {public_key[1]}"  
    )
```

SHETH L.U.J & SIR MV COLLEGE

```
private_key_box.insert(
    tk.END,
    f'd = {private_key[0]}\nn = {private_key[1]}"
)

result_label.config(
    text="Keys generated successfully",
    fg="green"
)

def sign():
    global signature

    if private_key is None:
        messagebox.showerror(
            "Error",
            "Generate keys first."
        )
        return

    message = message_box.get("1.0", tk.END).rstrip()

    if not message:
        messagebox.showerror(
            "Error",
            "Enter a message."
        )
        return

    signature = sign_message(message, private_key)

    signature_box.delete("1.0", tk.END)
    signature_box.insert(tk.END, str(signature))

    result_label.config(
        text="Message signed successfully",
        fg="green"
    )

def verify():
    if public_key is None:
        messagebox.showerror(
            "Error",
            "Generate keys first."
```

SHETH L.U.J & SIR MV COLLEGE

```
)  
return  
  
message = message_box.get("1.0", tk.END).rstrip()  
signature_text = signature_box.get("1.0", tk.END).strip()  
  
if not message:  
    messagebox.showerror(  
        "Error",  
        "Enter a message."  
    )  
    return  
  
if not signature_text:  
    messagebox.showerror(  
        "Error",  
        "Generate a signature first."  
    )  
    return  
  
try:  
    signature_value = int(signature_text)  
except ValueError:  
    messagebox.showerror(  
        "Error",  
        "Invalid signature."  
    )  
    return  
  
if verify_signature(  
    message,  
    signature_value,  
    public_key  
):  
    result_label.config(  
        text="VALID SIGNATURE\nMessage integrity and authenticity verified",  
        fg="green"  
    )  
else:  
    result_label.config(  
        text="INVALID SIGNATURE\nMessage has been modified or signature is invalid",  
        fg="red"  
    )
```

SHETH L.U.J & SIR MV COLLEGE

```
root = tk.Tk()
root.title("T071-Digital Signature")
root.geometry("800x700")
root.configure(bg="#f2f2f2")

title = tk.Label(
    root,
    text="T071-DIGITAL SIGNATURE",
    font=("Arial", 22, "bold"),
    bg="#f2f2f2",
    fg="#222222"
)
title.pack(pady=15)

message_label = tk.Label(
    root,
    text="Message",
    font=("Arial", 13, "bold"),
    bg="#f2f2f2"
)
message_label.pack()

message_box = tk.Text(
    root,
    height=5,
    width=85,
    font=("Arial", 11)
)
message_box.pack(pady=5)

button_frame = tk.Frame(
    root,
    bg="#f2f2f2"
)
button_frame.pack(pady=10)

generate_button = tk.Button(
    button_frame,
    text="Generate Keys",
    command=generate,
    width=18,
    bg="#3498db",
    fg="white",
    font=("Arial", 11, "bold")
)
```

SHETH L.U.J & SIR MV COLLEGE

```
)  
generate_button.grid(row=0, column=0, padx=5)
```

```
sign_button = tk.Button(  
    button_frame,  
    text="Sign Message",  
    command=sign,  
    width=18,  
    bg="#27ae60",  
    fg="white",  
    font=("Arial", 11, "bold")  
)
```

```
sign_button.grid(row=0, column=1, padx=5)
```

```
verify_button = tk.Button(  
    button_frame,  
    text="Verify Signature",  
    command=verify,  
    width=18,  
    bg="#8e44ad",  
    fg="white",  
    font=("Arial", 11, "bold")  
)
```

```
verify_button.grid(row=0, column=2, padx=5)
```

```
public_label = tk.Label(  
    root,  
    text="Public Key",  
    font=("Arial", 12, "bold"),  
    bg="#f2f2f2"
```

```
)  
public_label.pack()
```

```
public_key_box = tk.Text(  
    root,  
    height=4,  
    width=85,  
    font=("Arial", 9)
```

```
)  
public_key_box.pack(pady=5)
```

```
private_label = tk.Label(  
    root,  
    text="Private Key",
```

SHETH L.U.J & SIR MV COLLEGE

```
font=("Arial", 12, "bold"),
bg="#f2f2f2"
)
private_label.pack()

private_key_box = tk.Text(
    root,
    height=4,
    width=85,
    font=("Arial", 9)
)
private_key_box.pack(pady=5)

signature_label = tk.Label(
    root,
    text="Digital Signature",
    font=("Arial", 12, "bold"),
    bg="#f2f2f2"
)
signature_label.pack()

signature_box = tk.Text(
    root,
    height=5,
    width=85,
    font=("Arial", 9)
)
signature_box.pack(pady=5)

result_label = tk.Label(
    root,
    text="Generate keys to begin",
    font=("Arial", 12, "bold"),
    bg="#f2f2f2",
    fg="blue"
)
result_label.pack(pady=15)

root.mainloop()
```

SHETH L.U.J & SIR MV COLLEGE

Output:

T071-Digital Signature

T071-DIGITAL SIGNATURE

Message

gazala

Generate Keys Sign Message Verify Signature

Public Key

e = 65537
n = 8349314827314217524039915046765550090308637815675204593721961814120316921152668533
458062666194072554005155351703648718876918538529416576259230815157161761

Private Key

d = 4495892705737503035283406350616541231472173406093931216144285402449089585233855988
486271654373296549031330284425856323748445500793597957752721597622681473
n = 8349314827314217524039915046765550090308637815675204593721961814120316921152668533
458062666194072554005155351703648718876918538529416576259230815157161761

Digital Signature

4030574016032146059931787611948450710046438799539955896001825546909740074757018293052
167006243255329661717991582265691662970976677096648932766670752893032

VALID SIGNATURE
Message integrity and authenticity verified

Windows taskbar: Type here to search, 29°C Mostly cloudy, 23:00 10-08-2026

GAZALA ALAM

GAZALA ALAM
T071